

Reporte_Inventario

REPORTE DE PROYECTO

SISTEMA DE INVENTARIO POO

Materia: Programación Orientada a Objetos

Equipo: Leonardo arturo carmona vargas

1. Repositorio en GitHub

El código fuente, historial de versiones y ramas de desarrollo de los colaboradores se encuentran alojados de forma pública en la siguiente dirección:

<https://github.com/bsccarmona4-cyber/scholl-project.git>

2. Descripción del Sistema

Se implementó un mini sistema de control de inventarios físico desarrollado en Python 3, acoplado con una base de datos SQLite local que no requiere acceso a internet para funcionar. El sistema permite realizar altas de productos, reabastecimientos de stock, listar inventarios y registrar ventas con validaciones automáticas.

3. Conceptos de POO Aplicados

- **Abstracción:** Se definió la clase abstracta base 'ItemInventario' (heredando de ABC) para establecer el método abstracto obligatorio 'describir()'.
- **Herencia:** La clase 'Producto' hereda de 'ItemInventario', adquiriendo su constructor y atributos comunes de nombre y precio.
- **Encapsulación:** Los atributos del producto son privados (__stock, __id, __precio) y su consulta y modificación están estrictamente reguladas por decoradores '@property' y '@setter' con filtros lógicos.
- **Polimorfismo:** La clase 'Producto' da su propia forma concreta al método 'describir()', sobrescribiendo el comportamiento abstracto original.

4. Conexión a Base de Datos

La conexión a SQLite se gestiona mediante un context manager con '@contextmanager' que garantiza la liberación inmediata de los recursos del sistema, cerrando la conexión al archivo de base de datos automáticamente al

finalizar el bloque, incluso si ocurre una excepción de base de datos. Las consultas están parametrizadas con '?' para evitar vulnerabilidades de inyección de código SQL.

5. Control de Excepciones

Se estructuró una jerarquía limpia de excepciones heredando de Exception y ValueError:

- **PrecioInvalidoError (subclase de ValueError):** Se lanza cuando el precio ingresado es menor o igual a cero.
- **StockInsuficienteError (subclase de Exception):** Se lanza al intentar vender una cantidad que supera las existencias físicas.
- **ProductoNoEncontradoError (subclase de Exception):** Se lanza al intentar operar un ID inexistente en la BD.

6. Pruebas Unitarias

Se implementó una batería de 41 pruebas unitarias usando la biblioteca estándar 'unittest'. Los casos cubren flujos exitosos, control de errores lógicos y validaciones de datos basura. Para asegurar que las pruebas no alteren la base de datos real, cada prueba crea un archivo de base de datos temporal aislado usando la librería 'tempfile', el cual se destruye al finalizar la prueba.

7. Casos de Prueba de la Demo

Se verificaron con éxito los tres casos requeridos por la evaluación:

- **Caso 1 (Precio Negativo):** El intento de instanciar un producto con precio negativo lanza correctamente PrecioInvalidoError.
- **Caso 2 (Venta sin Stock):** El intento de vender 10 unidades cuando solo hay 3 en existencia lanza correctamente StockInsuficienteError.
- **Caso 3 (Desconexión de Internet):** Al emplear una base de datos SQLite embebida localmente, el sistema opera con normalidad sin presentar caídas de servicio.